Assign 1

과제 풀이 정리 · Notion 아카이브

5a÷문장 →Feature(특징)추출 - key = lambda x: (len(x),x)

|text| -> shortest word

==========

```
def find_shortest_lexicographically_first_word(text:str) -> str:
return min(text.split(), key=lambda x:(len(x),x),default="")
```

==========

text.split() : |text| -> |list| 반환 (공백을 기준으로 나눔)

=> [list] 중 최솟값 -> shortest word

어떤 것을 기준으로 최소?

-> 기준 = lambda : 표현식을 기준으로

다중 조건 정렬

key = lambda x: (len(x),x)

=> 기준 1. len(x) (길이 짧은 순)

=> 기준 2. x (사전 순서)

lambda x: (priority(x), x)

=> priority 기준으로 분류.

---

default=""

=> text가 Null일 때 디폴트값

5b÷Euclidean Distance(유클리드 거리)구현

position1, position2 -> 유클리드 거리

==========

```
def euclidean_distance(loc1: Position, loc2: Position) -> float:
  return ((loc1[0]-loc2[0])**2 + (loc1[1]-loc2[1])**2)**(1/2)
```

==========

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

5c÷문장 변형 - defaultdict(set)

문장 -> list of all "similar"

"similar"?

- same number of words

- pair of adjacent words

=> original에서 발생했던 인접 단어만 쓸 수 있음

- first word/last word 같음

EX>

Input : 'the red the blue the end'

Output : ['the red the blue the end',

'the blue the red the end',

'the red the red the end'

'the blue the blue the end']

==========

```
def mutate_sentences(sentence: str) -> List[str]:
words: list[str] = sentence.split()
if not words:
return []

//1 그래프 구성 -> {(str,Set[str]), ( , ), ( , ), ...}

graph: DefaultDict[str, Set[str]] = collections.defaultdict(set)
for i in range(len(word)-1):
graph[words[i].add(words[i+1])]

target_length = len(words)
end_word = words[-1]
result = []

//2 깊이 우선 탐색(DFS) 길이가 original이랑 같은 경우



def dsf(current_patt: List[str]) -> None:
if len(current_path) == target_length:
if current_path[-1] == end_word:
                    result.append("".joint(current_path))
return       //=> 길이 이상으로 탐색 X
```

current_word = current_path[-1]

for next_word in grapth[current_word]:

current_path.append(next_word)

dfs(current_path)

current_path.pop() //=> stack 느낌.

dfs([words[0]])

return result

==========

5d÷dot_product

v1, v2: Sparse Vector -> dot_product

Sparse Vector : vlaue = 0이면 key도 없는 vector

==========

def sparse_vector_dot_product(v1: SparseVector, v2: SparseVector) -> float:

if len(v1) > len(v2):

v1, v2 = v2, v1 //=> 항상 len(v2)>len(v1)

return sum(v * v2.get(k,0) for k,v in v1.items())

==========

dot product

=> 같은 key를 가진 것 끼리 곱하고

-> 곱한 결과를 합하면 됨.

---

sum(v * v2.get(k,0) for k,v in v1.items())

에서 v = v1.value

v2.get(k,0) = v2.value

get(k,0) : k에 해당하는 value를 가져오되,

없으면, 0을 반환

5e÷increment

|v1|, scale, |v2|

-> perform : v1 = v1 + scale * v2

weigth update 할 때 쓰이는 연산

(v1: weight, v2: features)

==========

```
def increment_sparse_vector(v1: SparseVector, scale: float, v2: SparseVector) -> None:
    if scale == 0:
      return
    for key, value in v2.items():
      v1[key] = v1[key] + scale * value
```

5f ~~:~~most_frequent_words - `defaultdict(int)`

```
문장 -> most_frequent_words
==================================================
def find_most_frequent_words(text: str) -> Set[str]:
if not text:     //text가 없으면
  return set()

  counts = collections.defaultdict(int)
  for word in text.split():
  counts[word] += 1

  max_value = max(counts.values())

  # 딕셔너리의 아이템을 순회하며 최댓값과 일치하는 단어만 집합으로 묶어 반환
  return {word for word, count in counts.items() if count == max_value}

==================================================
collections.defaultdict(int)
->디폴트 딕트를 만듦
(int) => 디폴트 값 = int()이 됨
=> int() -> 0 반환
```

← 인공지능 · 포스텍 목차로